Smart Industrial Management System

A Web-Based Centralized Platform for Machine, Worker, Production & Inventory Management

(ES-451 : Minor Project)

Submitted in partial fulfillment of the requirement
for the award of the degree of

Bachelor of Technology

in

Computer Science & Technology

Submitted By

Rakesh Gupta (16014812723)

Ankit Rana (15514812723)

Aditya Jaiswal (15614812723)

Under the supervision of

Dr. Garima

Professor (CST)



COMPUTER SCIENCE AND TECHNOLOGY

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

ROHINI SEC-22, NEW DELHI – 110086

Dec 2026

DECLARATION

This is to certify that the material embodied in this Minor Project titled “Smart Industrial Management System: A Web-Based Centralized Platform for Machine, Worker, Production & Inventory Management” being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Technology is based on our original work. It is further certified that this Minor Project work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. Our indebtedness to other works has been duly acknowledged at the relevant places.

(Rakesh Gupta)	(Ankit Rana)	(Aditya Jaiswal)
16014812723	15514812723	15614812723

CERTIFICATE

In this Minor Project titled "**Smart Industrial Management System: A Web-Based Centralized Platform for Machine, Worker, Production & Inventory Management**". This is to certify that the work embodied, being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Technology, is original and has been carried out under my supervision by,

Rakesh Gupta (Enrollment no. 16014812723), Ankit Rana (Enrollment no. 15514812723), Aditya Jaiswal (Enrollment no. 15614812723) under my supervision and guidance.

It is further certified that this Minor Project work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

(Dr. Garima)

Professor (CST)

(Dr. Pooja Gupta)

HOD (CST)

Maharaja Agrasen Institute of Technology

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Garima, Professor (CST), for her valuable guidance, constructive feedback and constant encouragement throughout the planning and development of this minor project. Her suggestions helped us organise a large problem into well-defined modules, keep the scope realistic, and present the work honestly, separating what is implemented from what is proposed.

We are thankful to Dr. Pooja Gupta, Head of the Department, Computer Science and Technology, and to Maharaja Agrasen Institute of Technology for providing the academic environment, laboratory facilities and resources needed to complete this work. We also thank all the faculty members of the CST department for their cooperation and support.

We acknowledge the authors and maintainers of the open-source tools on which the system is built, in particular the documentation of React, Node.js, Express, MongoDB, Mongoose, Socket.io, Recharts, Tailwind CSS, FastAPI and scikit-learn, as well as the creators of the public AI4I 2020 predictive maintenance dataset, which we used to test our machine-learning approach on data we did not generate ourselves.

Finally, we thank our peers, families and friends for their motivation, patience and support throughout the project.

ABSTRACT

Small and medium industrial units often run their machines, workers, production orders and stock on registers, spreadsheets and disconnected tools. A delay recorded in one place is invisible in another, maintenance is reactive, and managers cannot see the state of the whole floor without walking it. The Smart Industrial Management System (SIMS) is a web-based platform that brings these four areas into one real-time application with role-based access for administrators, managers and workers.

SIMS is built as a React and TypeScript client, a Node.js and Express REST API with Socket.io for live updates, MongoDB for storage, and a separate Python FastAPI service for machine learning. Machine status changes write downtime logs and raise alerts; production orders follow a validated state machine and consume materials atomically; stock movements are guarded so quantities never go negative and low stock drafts a purchase order that only a manager can approve. A dashboard and analytics pages present downtime, output, stock, attendance, reliability and OEE figures calculated by the backend.

The intelligence layer has two parts. A machine-learning service scores failure risk, checks telemetry for anomalies and forecasts stock cover and order completion. An LLM-based layer lets a manager ask operational questions in plain English: the application selects fixed, allow-listed database queries, passes only the retrieved figures to the language model, and rejects any answer whose numbers are not present in that evidence.

The report documents the problem, requirements, design and implementation, and reports measured results only: 438 automated tests pass (269 server, 113 client, 56 ML service), API responses had a median of 3.9 to 7.0 ms on a local machine, and live updates reached a second client in a median of 7.8 ms. The risk model is trained on synthetic data and is not presented as validated on real failures.

Keywords: Industrial Management, Predictive Maintenance, Real-Time Dashboard, Socket.io, MERN, Role-Based Access Control, Machine Learning, LLM-Based Insights, Inventory Control.

TABLE OF CONTENTS

Chapter 1: Introduction **1**
1.1 Background 1
1.2 Introduction to Smart Industrial Management System 1
1.3 Need for the Project 1
1.4 Aim and Scope 2
1.5 Existing System 2
1.6 Proposed System 2
1.7 Importance of Industrial Automation 2
1.8 Organisation of the Report 3
Chapter 2: Problem Statement **4**
2.1 Problem Definition 4
2.2 Core Challenges 4
2.3 Objectives 4
2.4 Expected Outcomes 5
2.5 Project Scope 5
2.6 Challenges and Constraints 5
Chapter 3: Analysis **6**
3.1 Software Requirement Specifications 6
3.2 Feasibility Study of the Project 8
3.3 Hardware and Software Requirements 9
3.4 Tools, Technologies and Platform Used 9
3.5 Use Case Diagram 10
3.6 Data Flow Diagrams 11
3.7 Input, Processing and Output Analysis 12
Chapter 4: Design and Architecture **13**
4.1 Overall System Architecture 13
4.2 Structure Chart and Work Breakdown Structure 14
4.3 Explanation of Modules 14
4.4 Frontend Architecture 15
4.5 Backend Architecture 15
4.6 Class Diagram 15
4.7 Flow Chart and Activity Diagram 16
4.8 Real-Time Communication Architecture 16
4.9 Database Design 17
4.10 AI/ML Architecture 18
Chapter 5: Implementation **19**
5.1 Implementation Overview 19
5.2 Authentication and Role-Based Access 19
5.3 Screenshots of the Application 20
5.4 Source Code of Some Modules 24
5.5 AI/ML Integration Summary 26
Chapter 6: Testing and Results **27**
6.1 Testing Methodology 27
6.2 Automated Test Results 27
6.3 Sample Test Cases 28
6.4 Measured Performance 28
6.5 Evaluation of the ML and AI Components 29
6.6 Security Checks 29
6.7 Frontend and Usability Checks 29
Chapter 7: Conclusion and Summary **30**
7.1 Summary of Work 30
7.2 Objectives Achieved 30
7.3 Implemented and Proposed Features 30
7.4 Learning Outcomes 30
Chapter 8: Limitations and Future Scope **31**
8.1 Limitations 31
8.2 Privacy and Ethical Considerations 31
8.3 Future Scope 31
Bibliography **32**

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Use case diagram of the Smart Industrial Management System	10
3.2	DFD Level 0 – context diagram	11
3.3	DFD Level 1	11
3.4	DFD Level 2 – inventory and purchasing (process 5.0)	12
4.1	Overall system architecture	13
4.2	Structure chart	14
4.3	Work breakdown structure	14
4.4	Class diagram of the main domain entities	15
4.5	Activity diagram – changing a machine status	16
4.6	Sequence of a live status update from a manager to a worker's screen	16
4.7	Entity relationship diagram (main collections and their key fields)	17
4.8	AI/ML workflow – query path and machine-learning path	18
5.1	JWT authentication middleware and role guard (server/src/middleware/auth.js)	19
5.2	Sign-in page (demo accounts are pre-filled only in development builds)	20
5.3	Manager dashboard – KPI tiles, live machine floor and open alerts	20
5.4	Machines – status, AI risk, ML model band, health score and next service; detail panel with anomaly check	21
5.5	Risk panel of a machine	21
5.6	Workers – 30-day attendance heatmap and the day's attendance table	22
5.7	Production – kanban board with a bottleneck flag and projected finish for each order	22
5.8	Inventory – stock bars against reorder levels, days of cover forecast and CSV tools	23
5.9	New production order with the feasibility check	23
5.10	Machine status service (server/src/modules/machines/machine.service.js)	24
5.11	Authenticated socket connection, rooms and the emit helper (server/src/sockets/index.js)	24
5.12	Guarded stock update (server/src/modules/inventory/inventory.service.js)	25
5.13	Allowed order transitions and the start-order checks (server/src/modules/production/order.service.js)	25
5.14	Building features and scoring through the ML service (server/src/modules/ml/ml.service.js)	25
5.15	Detecting numbers that are not in the evidence (server/src/modules/ai/analyst.service.js)	26
5.16	Socket events that refresh the client cache (client/src/hooks/useLiveUpdates.ts)	26
6.1	Test runner output (purchasing)	27
6.2	pytest output (ML insights)	27

LIST OF TABLES

Table No.	Title	Page No.
1.1	Size of the delivered implementation (measured from the source tree)	3
2.1	Core challenges and how SIMS responds to each	4
2.2	Measurable targets set for the prototype and what the implementation shows	5
2.3	In scope and out of scope	5
3.1	Functional requirements – access, machines and workers	6
3.2	Functional requirements – production, inventory, analytics, platform and AI	7
3.3	Non-functional requirements	7
3.4	Role-based permissions implemented in the API	8
3.5	Development and runtime requirements	9
3.6	Technology stack, checked against the source code	9
3.7	Input, processing and output of the main modules	12
4.1	Major modules, their responsibilities and main endpoints	14
4.2	The 26 collections grouped by area	17
5.1	AI and ML features, where they appear and what they use	26
6.1	Automated tests by area	27
6.2	Selected functional test cases	28
6.3	API response time, 40 requests each (milliseconds)	28
6.4	Held-out evaluation of the learned models	29
7.1	Status of the features proposed in the synopsis	30

CHAPTER 1: INTRODUCTION

1.1 Background

Industrial output depends on four things working together: machines that stay available, workers who are in the right place, production orders that move forward, and materials that are in stock when they are needed. Large manufacturers run enterprise platforms such as SAP, Oracle Manufacturing or Siemens Opcenter to coordinate these. Small and medium enterprises, which make up most of India's manufacturing sector, rarely can: such platforms need licences, dedicated IT staff and long deployments. Many units therefore continue with paper registers and separate spreadsheets.

The cost of this fragmentation is practical. A stoppage written in a production register is not seen by the stores clerk, who keeps ordering material on the old schedule. A machine can run past its service date because nothing flags it until it breaks mid-shift. A supervisor cannot tell whether today's target will be met without walking the floor. Each gap is small, but together they push decisions to be late and reactive.

1.2 Introduction to Smart Industrial Management System

The Smart Industrial Management System (SIMS) is a centralised, web-based platform that unifies machine monitoring, worker management, production tracking and inventory control behind one real-time dashboard. Every module shares the same data store and the same event stream, so a change made in one place, for example a machine going offline, immediately updates the alerts, the production bottleneck flags and every connected screen.

SIMS adds an artificial-intelligence layer on top of this operational data. A separate machine-learning service estimates the risk of machine failure, detects unusual sensor readings and forecasts stock and order progress, while an LLM-based assistant answers questions such as "which machine had the most downtime this week?" in plain English using figures calculated by the backend.

1.3 Need for the Project

- Replace paper and spreadsheet records with one source of truth that every role can see in real time.
- Move maintenance from reactive repair to planned, data-informed action using downtime history, telemetry and risk scores.
- Keep inventory lean: detect low stock immediately, draft the reorder and keep a person in control of approving it.
- Give non-technical managers a way to ask questions of the data without building reports by hand.
- Enforce who may see and change what, with a record of every change, in a way that suits a single factory.

1.4 Aim and Scope

The aim is to design and build a working, tested prototype of a unified industrial management platform covering five modules: Machine Monitoring, Worker Management, Production Tracking, Inventory Management and an Analytics Dashboard, together with a machine-learning service and an LLM-based natural-language insights layer.

The scope is a single-factory, single-tenant web application. Machine status and sensor values are simulated or entered manually because no physical IoT hardware is connected. The failure-risk model is trained on synthetic data to demonstrate the pipeline end to end; it is an additional signal and not a validated predictor of real failures. Native mobile applications, multi-factory scaling and fully autonomous AI decisions are outside the scope.

1.5 Existing System

In the typical small industrial unit the existing system is a combination of handwritten registers, spreadsheets kept by different departments, and messaging apps. Data is updated late and entered more than once, so figures disagree between departments. There is no common alert path, no history of why a machine stopped, no automatic link between consumption and stock, and no single place where management can see key indicators. Enterprise platforms solve these problems but are too costly and heavy to deploy for most small units.

1.6 Proposed System

SIMS proposes a lightweight, modular alternative. It exposes a REST API and a live event channel, stores all records in MongoDB, and presents them through a responsive React interface in light and dark themes. The system writes downtime and stock history automatically as a side effect of normal actions, raises alerts when thresholds are crossed, restricts every action by role, and offers machine-learning and LLM features as optional layers that fail soft: if the ML service or the language model is unavailable, the rest of the application keeps working.

1.7 Importance of Industrial Automation

Industrial automation and Industry 4.0 aim to make production data available, trustworthy and actionable. Real-time visibility shortens the time between a problem occurring and someone acting on it; predictive maintenance reduces unplanned stoppages, which are the most expensive kind because they halt the whole downstream line; and accurate stock control lowers both over-ordering and stock-outs. Delivering these benefits through a web application running on ordinary hardware lowers the barrier for smaller units, which supports the UN Sustainable Development Goals 8 (decent work and economic growth), 9 (industry, innovation and infrastructure) and 12 (responsible consumption and production).

1.8 Organisation of the Report

This report is organised into eight chapters. **Chapter 1** introduces the background, the need for the project, its aim and scope, and the existing and proposed systems. **Chapter 2** states the problem, the core challenges, the objectives, the expected outcomes and the constraints. **Chapter 3** analyses the requirements, feasibility, technology stack, use cases and data flow. **Chapter 4** describes the architecture, modules, database design, activity flow and the AI/ML and real-time architecture. **Chapter 5** explains the implementation with screenshots of the running application and excerpts from the source code. **Chapter 6** presents the testing approach, test cases and measured results. **Chapter 7** summarises the work and the learning outcomes, and **Chapter 8** lists the limitations and the future scope. The bibliography follows.

Conventions used in this report

Wherever the report states a number it is either a requirement of the project or a value measured on the actual implementation, and the way it was measured is given. Features are described as implemented only if they exist in the source code and are covered by tests or were exercised in the running application; anything that is planned but not built is listed in Chapter 8. The AI components are described by their function and data flow; the choice of language-model provider is deliberately not discussed because it is a configuration detail that does not change the design.

Project at a glance

Table 1.1: Size of the delivered implementation (measured from the source tree)

Item	Measured value
Server (Node.js and Express) source lines, excluding tests	5,919
Client (React and TypeScript) source lines, excluding tests	5,943
ML service (Python) source lines, excluding tests	790
Automated test code lines (server, client and ML service)	5,869
MongoDB collections defined	26
REST operations documented in the API specification	132
Automated tests (server / client / ML service)	269 / 113 / 56
User roles	Admin, Manager, Worker

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

Industries face significant difficulty in managing their four operational pillars – machines, workers, production and inventory – efficiently. Most small and medium units depend on manual, paper-based or fragmented digital systems that operate in silos, with no single system talking to another. Real-time, unified visibility across the shop floor is therefore almost impossible, which leads to resource wastage, production delays, unplanned machine downtime and poor inventory control.

The problem is to design and build one connected, role-aware platform that records these four areas in a single database, keeps every user's view current without manual refresh, and adds machine learning and natural-language insight so that decisions become proactive rather than reactive.

2.2 Core Challenges

Table 2.1: Core challenges and how SIMS responds to each

Challenge	Description	Response in SIMS
Resource wastage	Labour, material and machine time are allocated from memory rather than current data.	Shared data for workers, machines and stock; feasibility check before an order is accepted.
Production delays	Separate registers hide bottlenecks until a deadline is at risk.	Order state machine, bottleneck flags, projected finish dates against due dates.
Machine downtime	Maintenance is reactive, so stoppages are unplanned and costly.	Downtime logs, health score, risk model, anomaly check, preventive schedules and work orders.
Inventory mismatch	Paper stock is updated late, causing over-ordering or surprise shortages.	Atomic stock transactions, reorder alerts, auto-drafted purchase orders, days-of-cover forecast.
Lack of visibility	No single place shows machine health, attendance, orders and stock together.	Live dashboard, analytics charts, notifications and a natural-language assistant.

2.3 Objectives

- To build one web platform that integrates machine monitoring, worker management, production tracking and inventory control.
- To provide real-time dashboards and automated alerts so that management monitors operational health from one place.
- To reduce unplanned downtime through a predictive-maintenance module driven by a machine-learning model.
- To make operational data accessible to non-technical managers through an LLM-based natural-language insights layer.
- To maintain lean inventory through live stock tracking and reorder triggers that keep a person in control.
- To implement secure role-based access for Admin, Manager and Worker, enforced on the backend.

2.4 Expected Outcomes

Table 2.2: Measurable targets set for the prototype and what the implementation shows

Target (from the synopsis)	Outcome in the implementation
One logged-in session shows live machine status, attendance, orders and stock without reload.	Achieved: the dashboard subscribes to Socket.io events and refreshes cached data (Chapter 5).
A status change on one client reaches another within 2–3 seconds.	Measured: median 7.8 ms, 95th percentile 13.3 ms over 20 trials on a local machine (Chapter 6).
The ML service returns a Low/Medium/High risk classification for a machine.	Achieved: risk band, probability and top contributing factors, shown on the Machines page.
A manager can ask a plain-English question and receive a grounded answer.	Achieved: answers are built from fixed queries and checked against the evidence before display.
Stock below a threshold raises a reorder alert.	Achieved: one alert per item, plus a draft purchase order.
Three roles see different views, enforced by JWT claims.	Achieved: roles are checked on every API route and socket connection.

2.5 Project Scope

Table 2.3: In scope and out of scope

In scope (this project)	Out of scope (future work)
Web dashboard for the five modules	Native mobile application
Simulated or manually entered machine status and readings	Physical IoT sensor integration on real machines
Failure-risk model on synthetic and public datasets	Production-grade model validated on proprietary plant data
Single-factory, single-tenant deployment	Multi-factory, multi-tenant SaaS
Three fixed roles with route-level permissions	Per-module custom permission editor
LLM answers grounded in live application data	Fully autonomous AI decisions such as auto-approving purchases

2.6 Challenges and Constraints

- **Data realism:** no real failure history was available, so the risk model uses synthetic data; a public dataset was used for an external check, and its limits are stated in Chapters 6 and 8.
- **Consistency across modules:** the same event, such as a machine stopping, must update logs, alerts, orders and screens together; this is why every module writes through a service layer that emits events after the database write.
- **Trust in AI output:** a language model can state invented numbers, so the design never lets it calculate or look up figures, and it verifies every number in its answer against the evidence.
- **Graceful failure:** the ML service and the language model are external dependencies, so every AI feature degrades to a clear message instead of failing the application.
- **Time:** the work was planned over eleven weeks with a working increment at the end of each phase.

CHAPTER 3: ANALYSIS

3.1 Software Requirement Specifications

The requirements were derived from the problem statement and refined module by module. They are grouped below as functional requirements, which describe what the system does, and non-functional requirements, which describe how well it must do it. Every requirement listed here has been implemented and is exercised by at least one automated test or by the running application.

3.1.1 Functional Requirements of the Project

Table 3.1: Functional requirements – access, machines and workers

ID	Requirement	Module
FR-01	Users sign in with e-mail and password; the server issues a short-lived access token (JWT) and a rotating refresh token kept in an httpOnly cookie.	Access
FR-02	Every API route and socket connection enforces one of three roles (Admin, Manager, Worker); a deactivated account or a token issued before a password reset is refused.	Access
FR-03	Admins create, edit, deactivate users and reset passwords; every create, update and delete is written to an audit log.	Access
FR-04	Failed sign-ins lock an account for 15 minutes after five misses; a single-use, 30-minute link resets a forgotten password.	Access
FR-05	Machines can be added, edited and removed; changing a status records a reason and closes and opens downtime logs automatically.	Machines
FR-06	Telemetry (temperature, vibration, rpm, power) is stored, compared with warning and critical limits and summarised as a 0–100 health score with every deduction listed.	Machines
FR-07	Maintenance work orders (corrective, preventive, inspection) follow a state machine; recurring schedules open preventive orders; an offline machine opens one corrective order.	Machines
FR-08	Reliability figures (failures, MTTR, MTBF, availability) are calculated per machine from the status logs.	Machines
FR-09	Workers and shifts are maintained; workers check in and out for themselves and are marked late after a grace period.	Workers
FR-10	Leave requests are filed by workers and approved or rejected by managers; approved leave is not counted as absence.	Workers
FR-11	Tasks are assigned to workers and moved through to-do, in-progress and done; shift handover notes are shared between shifts.	Workers

Table 3.2: Functional requirements – production, inventory, analytics, platform and AI

ID	Requirement	Module
FR-12	Production orders move only pending → in progress → completed; starting needs every assigned machine to be available.	Production
FR-13	A bill of materials per product is copied into new orders; starting an order consumes all materials in one all-or-nothing step.	Production
FR-14	Quality checks record inspected, defect and scrap counts; a feasibility check reports shortages before an order is accepted.	Production
FR-15	An order is flagged as a bottleneck when it is overdue or waits on an unavailable machine.	Production
FR-16	Stock movements (in, out, adjust) are atomic and can never take a quantity below zero; every movement is recorded.	Inventory
FR-17	Falling below the reorder level raises one alert per item and drafts a purchase order; a manager approves it and receiving it restocks.	Inventory
FR-18	Suppliers, stock valuation and CSV import and export are provided for inventory data.	Inventory
FR-19	KPI overview, downtime, output, stock, attendance, Pareto, reliability and OEE views are calculated by the backend, with CSV and print export.	Analytics
FR-20	Changes appear on every connected client without reload; notifications are stored per user and unacknowledged critical alerts are escalated.	Platform
FR-21	The ML service scores failure risk with a probability and top factors, flags anomalous telemetry and forecasts stock cover and order finish dates.	AI/ML
FR-22	A manager can ask a question in plain English and receive an answer built from database figures, with its sources shown; weekly summaries and alert explanations use the same grounding rules.	AI/ML

3.1.2 Non-functional Requirements of the Project

Table 3.3: Non-functional requirements

Attribute	Requirement and how it is met
Security	Hashed passwords, 15-minute access tokens, route-level role checks, request validation (Zod), input sanitising, rate limiting, security headers, audit log.
Performance	Common list and dashboard endpoints respond in milliseconds on a development machine; measured values are given in Chapter 6.
Reliability	The ML service and the language model are optional: if either is down the feature reports it and the rest of the application is unaffected.
Data integrity	Stock and order changes use guarded atomic updates; duplicate alerts, reorder drafts and work orders are prevented.
Usability	Responsive layout, light and dark themes, keyboard-accessible dialogs, toasts and a notification centre.
Maintainability	Route → service → model layering, one folder per module, 438 automated tests.
Scalability limit	Designed for one factory; no multi-tenant isolation or horizontal scaling of the socket layer.

Table 3.4: Role-based permissions implemented in the API

Resource	Admin	Manager	Worker
Users and audit log	Create, edit, deactivate, reset passwords; read audit log	Read user list	Own sign-in only
Machines, telemetry	Full, including status change	Full, including status change	Read
Maintenance work orders	Full	Full	Update status and notes of own assigned orders
Workers, shifts, leave	Full	Full, approve leave	Own attendance, own leave requests
Production orders, BOM	Full	Full	Read orders linked to own tasks; log quality checks for them
Inventory, purchasing	Full, approve purchase orders	Full, approve purchase orders	Read inventory
Analytics, AI analyst, ML	Full	Full	None
Alerts, handover notes	Full	Full	Read alerts; write and close own handover notes

3.2 Feasibility Study of the Project

3.2.1 Technical Feasibility

Every technology needed is mature, documented and runs on an ordinary laptop: Node.js and MongoDB for the application, React for the interface, Socket.io for live updates, and Python with scikit-learn for the ML service. The riskiest parts, real-time propagation and trustworthy AI output, were prototyped early and are covered by tests; both proved feasible.

3.2.2 Economic Feasibility

The stack is open source and there are no licence costs. Development used existing laptops, a local MongoDB instance and free tiers of hosting services. Running the language model has a usage cost, which the design limits by caching results, rate-limiting calls and making the feature optional. The report does not claim zero total cost: deployment, hosting and model usage would carry running costs.

3.2.3 Operational Feasibility

Users only need a web browser. The interface follows common patterns (tables, a kanban board, forms) and role-based navigation shows each person only the pages they may use. Managers can adopt the AI features gradually because the core screens work without them.

3.2.4 Schedule Feasibility

The work was planned as an eleven-week Agile cycle: requirements and design, database and backend, frontend, machine learning, real-time layer, AI integration, then testing and documentation, with a demonstrable increment at the end of each phase. This schedule was adequate for the five core modules plus the AI layer.

3.3 Hardware and Software Requirements

Table 3.5: Development and runtime requirements

Item	Specification
Processor	Intel Core i5 (10th generation) or equivalent, or higher
Memory	8 GB minimum; 16 GB recommended when the ML service runs alongside the application
Storage	256 GB SSD
Operating system	Windows 11 was used for development; any OS that runs Node.js 22 and Python 3 works
Runtime software	Node.js 22, MongoDB 8, Python 3.13 with the packages in requirements.txt
Client	A current browser (Chrome, Edge or Firefox); the app is installable as a progressive web app
Network	Internet access only for the optional language-model feature and for cloud hosting

3.4 Tools, Technologies and Platform Used

Table 3.6: Technology stack, checked against the source code

Layer	Technology	Purpose in SIMS
Frontend	React 19, TypeScript, Vite	Component-based single-page application with type-checked code
	Tailwind CSS 4, Recharts	Styling with light and dark themes; all charts (the synopsis also listed Chart.js, which was not needed)
	TanStack Query, Zustand, React Router	Server-state cache, small client stores (session, theme, toasts), role-based routes
	socket.io-client	Live events that refresh cached data
Backend	Node.js, Express 4	REST API with middleware, services and route modules
	JWT (jsonwebtoken), bcryptjs	Access and refresh tokens, password hashing
	Zod, helmet, express-rate-limit, mongo-sanitize	Validation, security headers, rate limits, input sanitising
	Socket.io, node-cron, pino	Real-time channel, scheduled jobs, structured logging
Database	MongoDB 8, Mongoose 8	Document storage for 26 collections with schemas and indexes
AI / ML layer	Python, FastAPI, scikit-learn	Risk, sensor, anomaly and forecasting models served over REST
	LLM-based intelligence integration	Natural-language question answering, summaries and explanations over backend-supplied evidence
Tools	Git, Postman collection, Swagger UI	Version control, API testing (collection generated from the API spec), API documentation
	node:test, Vitest, pytest	Automated testing of server, client and ML service

3.4.1 How the Technologies Work Together

A user action in the React interface calls the Express API through an Axios client that attaches the access token. The API validates the request, checks the role and calls a service function. The service writes to MongoDB through Mongoose and, only after the write succeeds, emits a Socket.io event; the browser receives the event and tells TanStack Query to refetch the affected data, so every open screen updates without a reload. Scheduled jobs call the same service functions, so a nightly or hourly job triggers the same events as a manual action.

For machine learning, the Node service assembles numeric features from the database and sends them to the FastAPI service, which returns a risk band, a probability and the factors that raised it; the result is stored and an alert policy decides whether to notify. For natural-language questions, the backend chooses fixed database queries, runs them, and sends only the retrieved figures to the language model, then verifies the reply before showing it. The same figures are shown beside the answer so the reader can check them.

3.5 Use Case Diagram

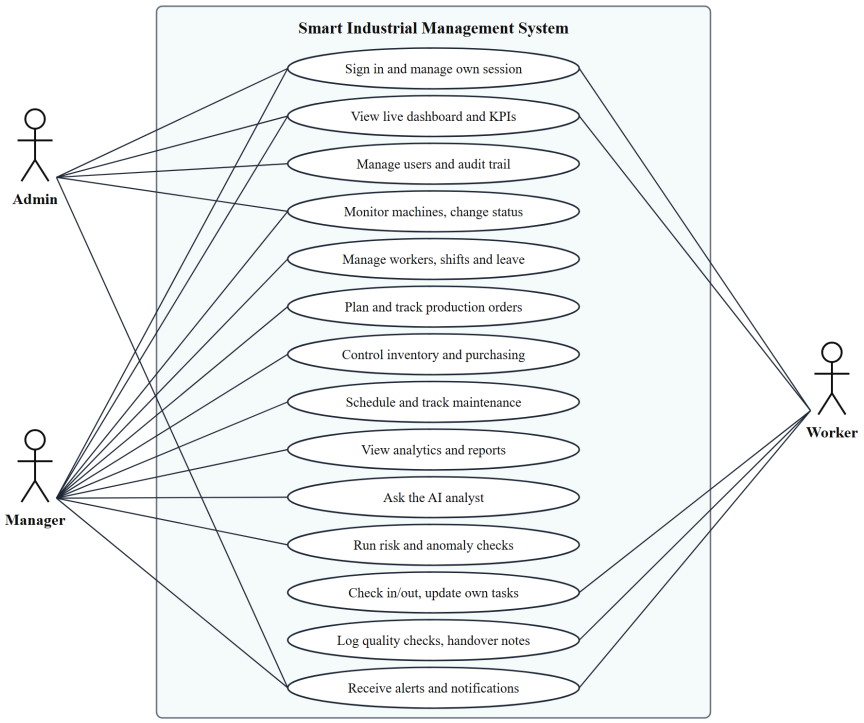


Figure 3.1: Use case diagram of the Smart Industrial Management System

Admins and managers share most use cases; admins additionally manage users and the audit trail. Workers have a deliberately small set: signing in, viewing the dashboard data they may read, checking in and out, updating their own tasks, logging quality checks and handover notes, and receiving alerts.

3.6 Data Flow Diagrams

The level 0 diagram shows SIMS as one process exchanging data with four external parties and one data store. Users and the ML and language-model services never touch the database directly; every read and write passes through the application.

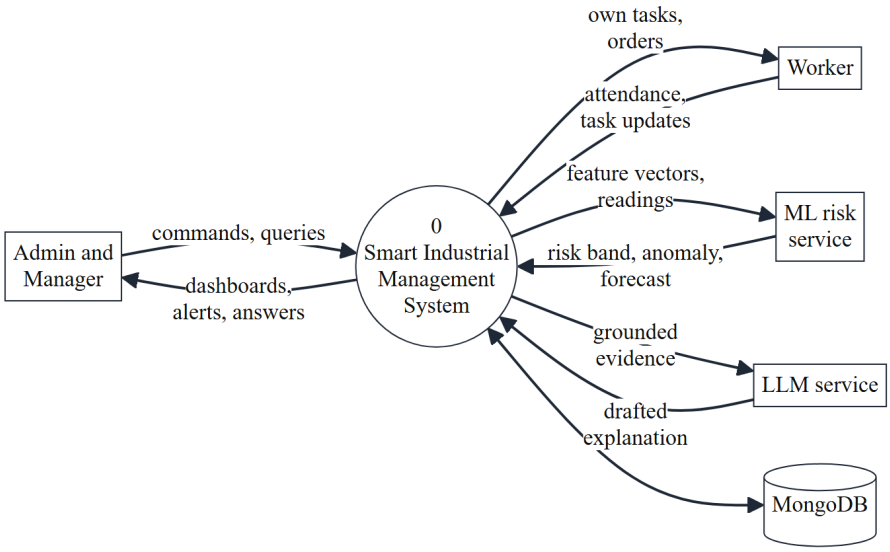


Figure 3.2: DFD Level 0 – context diagram

Level 1 breaks the system into seven processes and six data stores. Authentication guards the four operational modules, which feed analytics and alerts; the AI insights process only reads data.

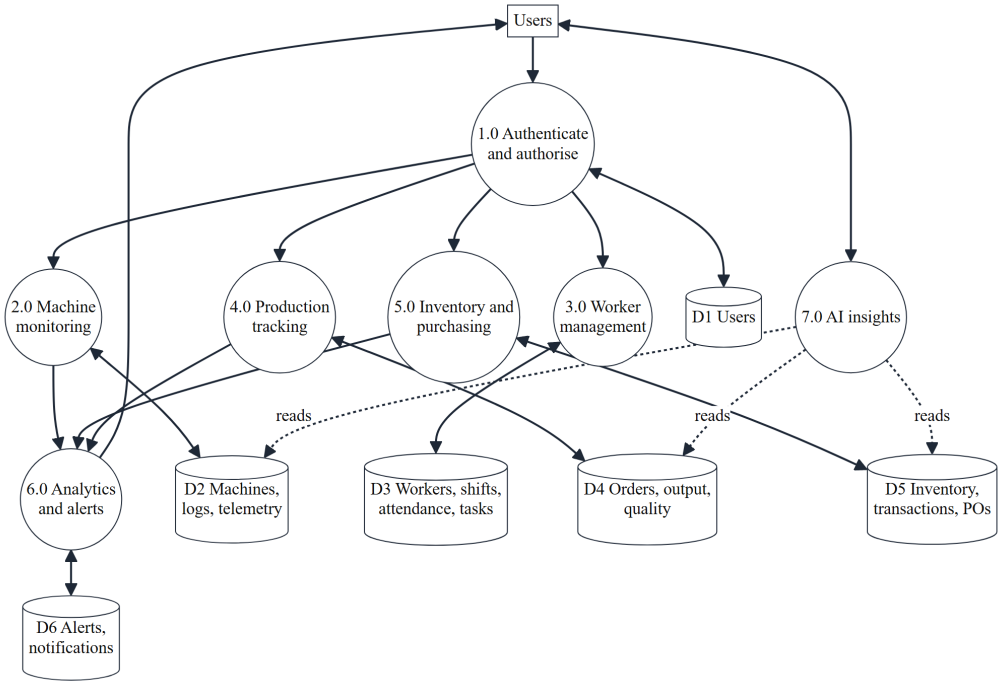


Figure 3.3: DFD Level 1

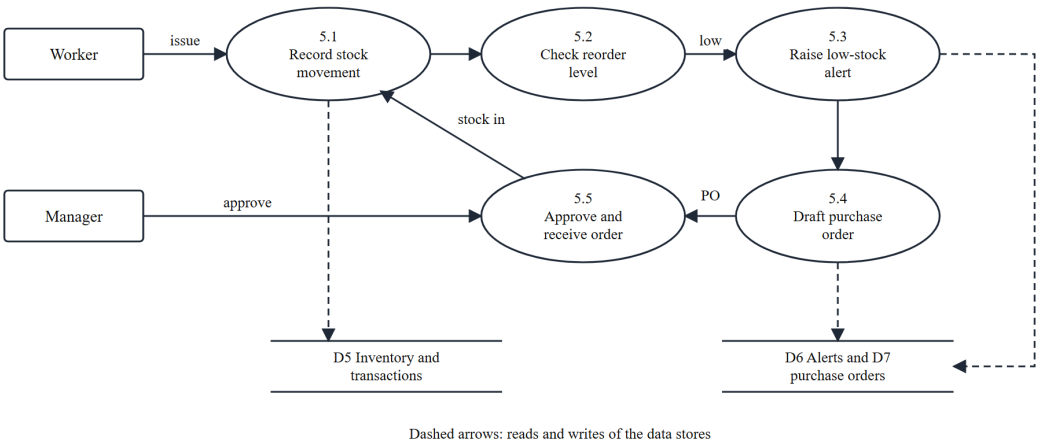


Figure 3.4: DFD Level 2 – inventory and purchasing (process 5.0)

Level 2 expands process 5.0. A stock movement is recorded atomically, the reorder level is checked, a low-stock alert is raised once, and a purchase order is drafted. Only a manager can approve it; receiving the approved order posts a stock-in movement and closes the loop.

3.7 Input, Processing and Output Analysis

Table 3.7: Input, processing and output of the main modules

Module	Input	Processing	Output
Machines	Status change with reason; sensor readings	Close and open downtime log; compare with limits; compute health score	Updated status, alerts, work order, live event
Workers	Check-in, leave request, task update	Late-grace rule, overlap check, role-owned updates	Attendance record, heatmap, notifications
Production	New order, start, output quantity	State machine, material consumption, bottleneck check, feasibility	Kanban board, progress, projected finish
Inventory	Stock in, out, adjust; CSV rows	Guarded atomic update; threshold check; PO drafting	Stock levels, alerts, draft purchase orders, valuation
Analytics	Date range (7, 14, 30 days)	Aggregation pipelines in MongoDB	KPI tiles, charts, Pareto, OEE, CSV
AI and ML	Question and period; numeric features	Fixed queries, model scoring, grounding check	Answer card with sources; risk band; forecast

CHAPTER 4: DESIGN AND ARCHITECTURE

4.1 Overall System Architecture

SIMS follows a four-tier design. The **client tier** is a React single-page application. The **application tier** is a Node.js and Express server that owns validation, authorisation, business rules and the real-time channel. The **data tier** is MongoDB, accessed only through Mongoose models. The **AI and ML tier** consists of a separate Python FastAPI service for the learned models and an external language-model service that is reached only by the intelligence layer of the application tier.

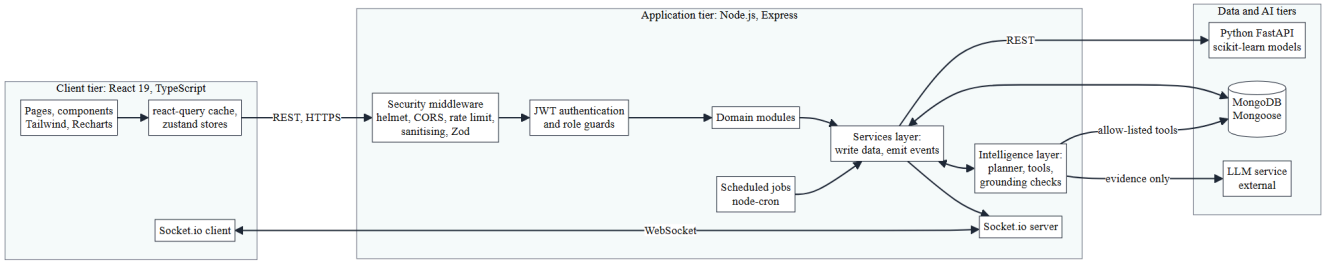


Figure 4.1: Overall system architecture

Request lifecycle

1. The browser sends a REST request with the access token; security middleware applies headers, CORS, rate limits and input sanitising.
2. The authentication middleware verifies the token and reads the user's current role and active flag, so a role change or deactivation takes effect within seconds.
3. The route validates the body with a Zod schema and checks the role; the handler calls a service function.
4. The service reads and writes MongoDB, then emits Socket.io events and, where needed, calls the ML service or the intelligence layer.
5. The reply uses one envelope, `{ success, data, message, errors }`, so the client handles every response the same way.

Design principles

- **Services own side effects.** Events, alerts and notifications are produced by services after a successful write, never by route handlers, so REST calls, scheduled jobs and the simulator all behave identically.
- **AI is optional and bounded.** The ML service and the language model sit behind timeouts and fall back to a clear message; neither can write to the database on its own.

4.2 Structure Chart and Work Breakdown Structure

The structure chart decomposes the system into seven functional areas, each with three main functions. The work breakdown structure shows how the project effort was divided into six work packages.

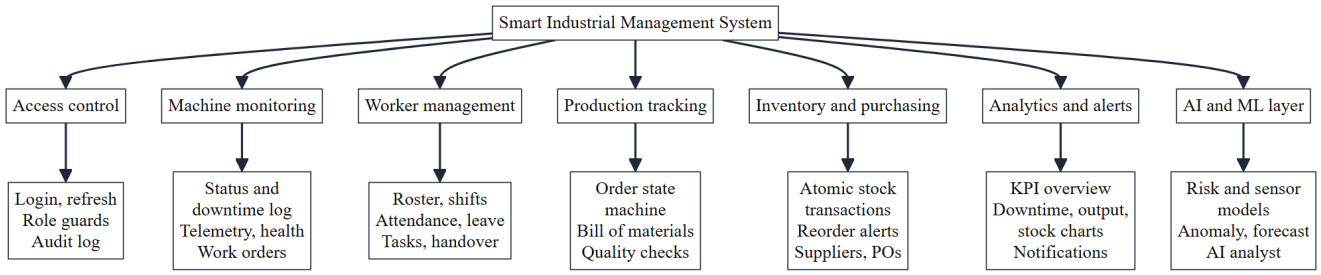


Figure 4.2: Structure chart

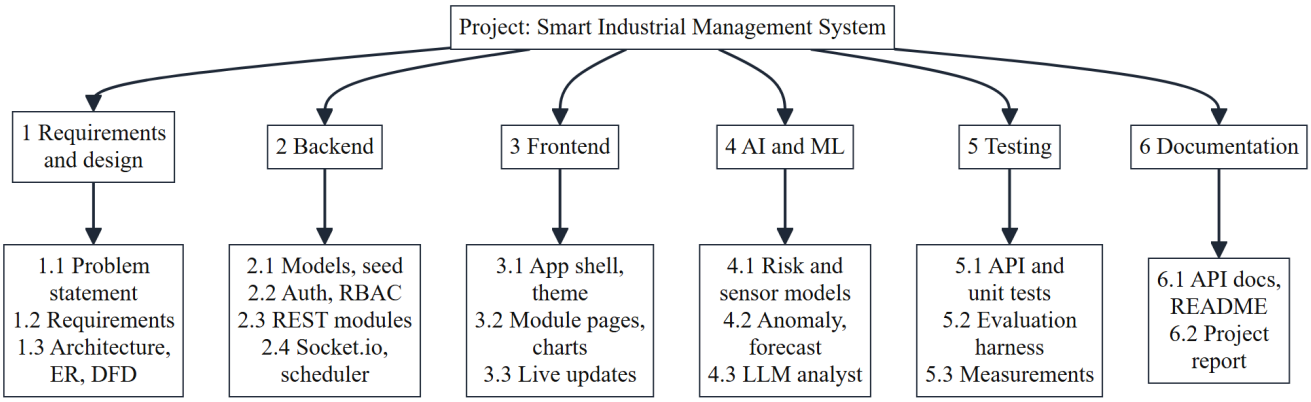


Figure 4.3: Work breakdown structure

4.3 Explanation of Modules

Table 4.1: Major modules, their responsibilities and main endpoints

Module	Responsibility	Main endpoints
Access	Sign-in, token refresh, lockout, password reset, user administration, audit log	/auth, /users, /audit-logs
Machines	Registry, status and downtime logs, telemetry, health score, work orders, schedules	/machines, /telemetry, /maintenance
Workers	Roster, shifts, attendance, leave, tasks, handover notes	/workers, /attendance, /leave, /tasks, /handover
Production	Orders, state machine, output, BOM, quality checks, feasibility	/orders, /bom, /quality-checks, /production
Inventory	Items, atomic transactions, valuation, import, suppliers, purchase orders	/inventory, /suppliers, /purchase-orders
Analytics and alerts	KPI aggregation, charts, alerts, notifications, escalation	/analytics, /alerts, /notifications
AI and ML	Risk and sensor models, anomaly, forecasts, analyst, summaries	/ml, /insights, /ai, /machines/:id/…

4.4 Frontend Architecture

The client is organised by function: `pages` (one per route, loaded on demand so the chart library is downloaded only when a chart page opens), `components` (shared interface pieces such as the application shell, dialogs, charts and the notification bell), `hooks` (the live-update hook), `store` (Zustand stores for the session, theme and toasts) and `lib` (the API client with silent token refresh, CSV helpers and shared types). Server data is cached by TanStack Query; the live-update hook listens to Socket.io events and invalidates the matching cache keys, which is how a change on one machine reaches every open screen. Navigation is built from the signed-in role, and routes meant for staff redirect workers to their own page.

4.5 Backend Architecture

The server is arranged as `routes → services → models` inside one folder per module. Routes only validate, authorise and delegate; services hold the business rules and are the only code that emits events; models define schemas and indexes. Cross-cutting concerns live in middleware: authentication with a short cache of each user's role, role guards, validation, rate limiting, audit logging and one error handler that turns every failure into the common response envelope. Four scheduled jobs run inside the process: hourly risk scoring, a daily check that opens preventive work orders, a ten-minute escalation of unacknowledged critical alerts, and a weekly summary.

4.6 Class Diagram

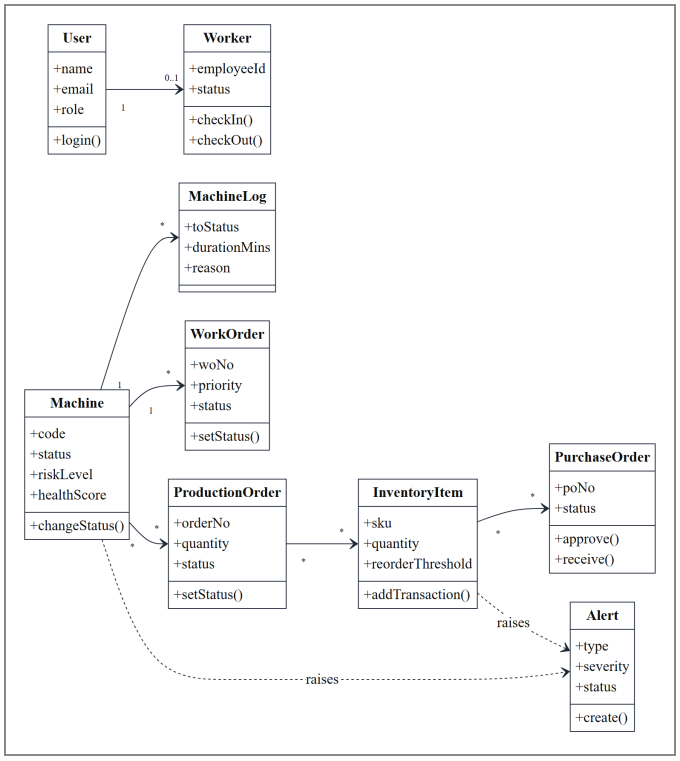


Figure 4.4: Class diagram of the main domain entities

JavaScript has no class declarations here, so the diagram models each Mongoose schema as a class and each service function that changes it as an operation.

Machine is the centre: it owns its status history (MachineLog) and its work orders, is assigned to production orders, and raises alerts. **ProductionOrder** consumes **InventoryItem** quantities when it starts. **InventoryItem** raises a low-stock alert and, through reorder drafting, appears on **PurchaseOrder** lines.

User owns the login; a **Worker** profile is linked to a user only for the Worker role. Cardinalities follow the references stored in the schemas.

4.7 Flow Chart and Activity Diagram

The swim-lane diagram follows one of the most important actions in the system: changing a machine's status. Errors are caught before any write; once the change is accepted, the service closes the old downtime log, opens a new one and emits one event that every client receives.

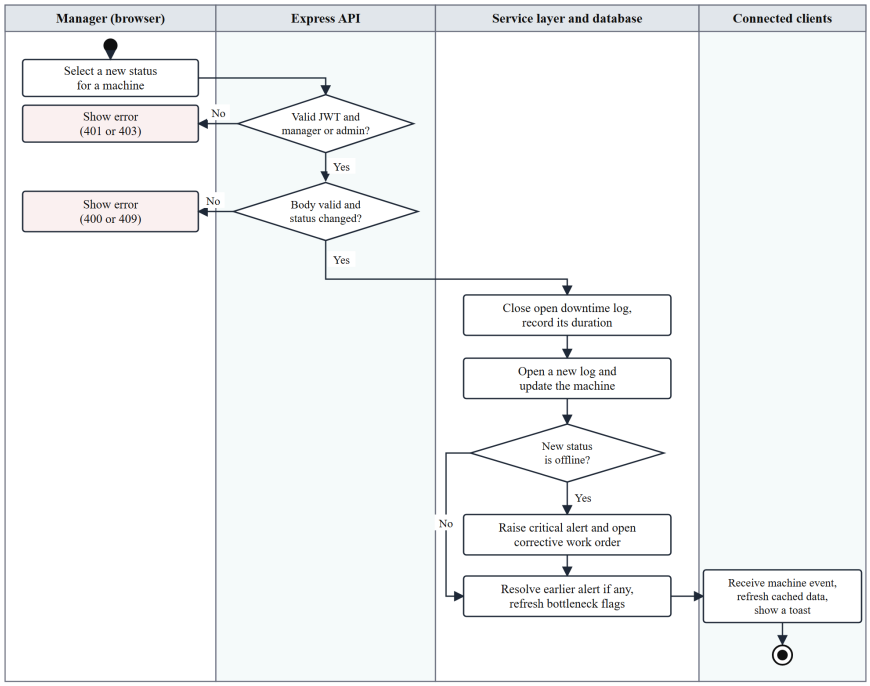


Figure 4.5: Activity diagram – changing a machine status

4.8 Real-Time Communication Architecture

Clients connect over WebSocket with the access token in the handshake, which is validated with the same rules as the REST API. Each connection joins rooms by role (`room:all`, `room:staff`), by user (`user:<id>`, used for personal notifications) and optionally by machine (`machine:<id>`).

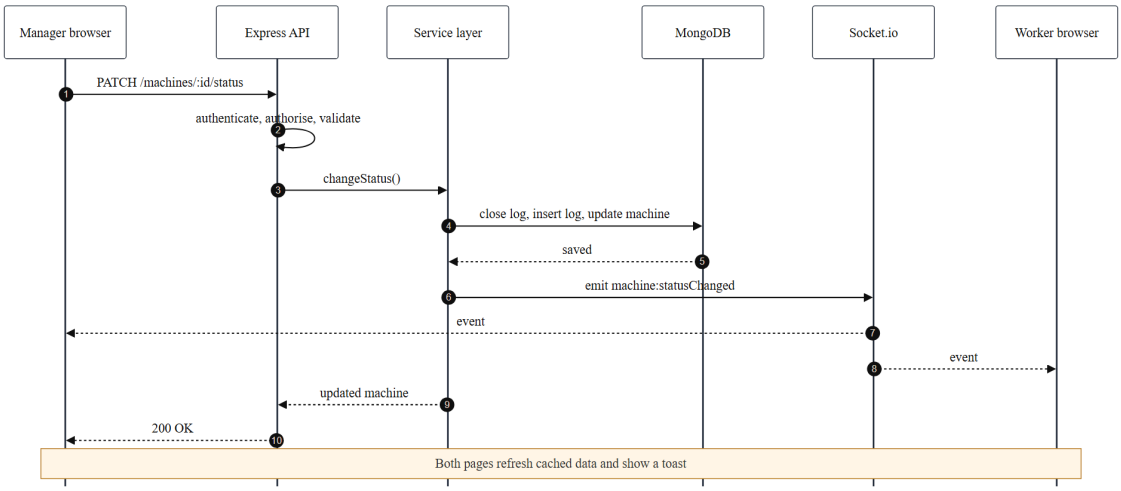


Figure 4.6: Sequence of a live status update from a manager to a worker's screen

4.9 Database Design

MongoDB stores 26 collections. References between documents use ObjectIds and are populated by Mongoose when a view needs them; counters that must stay correct under concurrency, such as stock quantity, are updated with atomic operators rather than read-modify-write. Indexes support the common queries: machine status, machine log by machine and time, unique worker-and-date for attendance, order status and due date, alert status and time, and notification by user.

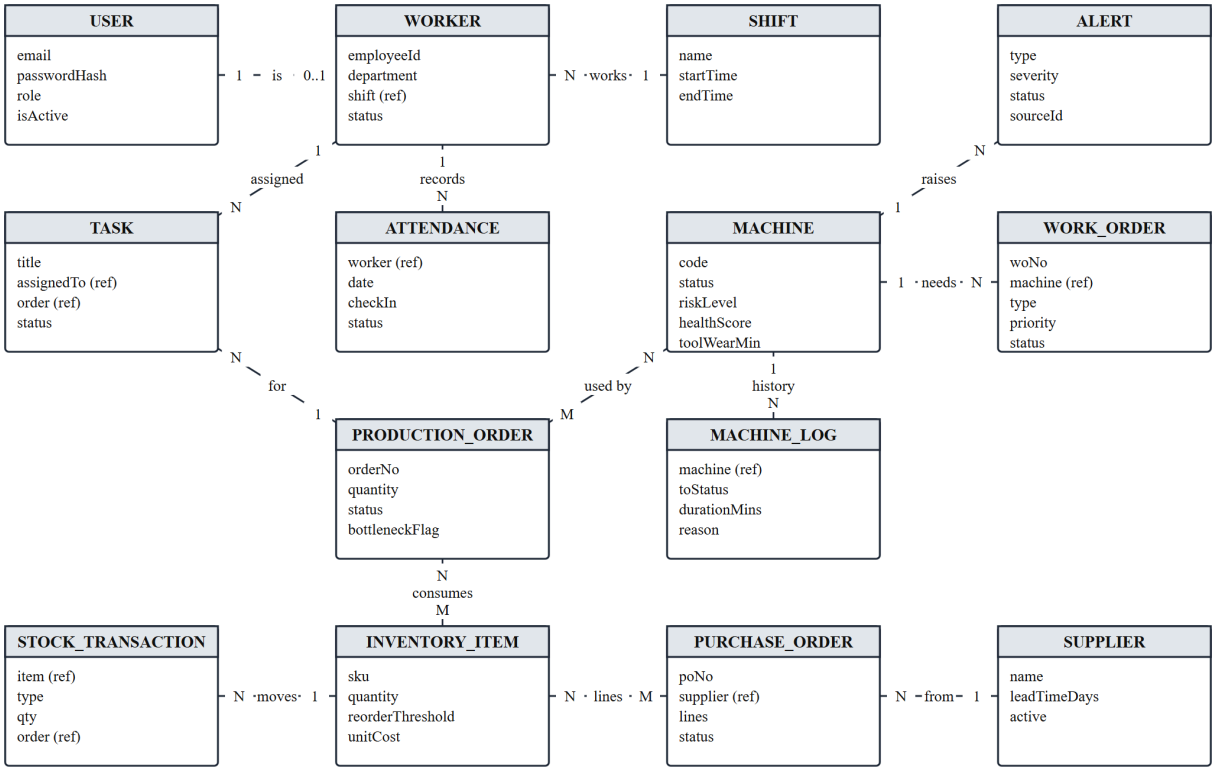


Figure 4.7: Entity relationship diagram (main collections and their key fields)

Table 4.2: The 26 collections grouped by area

Area	Collections
Access	User, AuditLog
Machines and maintenance	Machine, MachineLog, SensorReading (expires after 7 days), WorkOrder, MaintenanceSchedule, HandoverNote
Workforce	Worker, Shift, Attendance, Task, LeaveRequest
Production	ProductionOrder, OutputLog, BOM, QualityCheck
Inventory and purchasing	InventoryItem, StockTransaction, Supplier, PurchaseOrder
Alerts and AI	Alert, Notification, MachineAssessment, MlPrediction, AiReport

4.10 AI/ML Architecture

The intelligence of SIMS has two paths that share the same database but are otherwise independent: an LLM-based path for natural-language insight and a machine-learning path for numeric prediction.

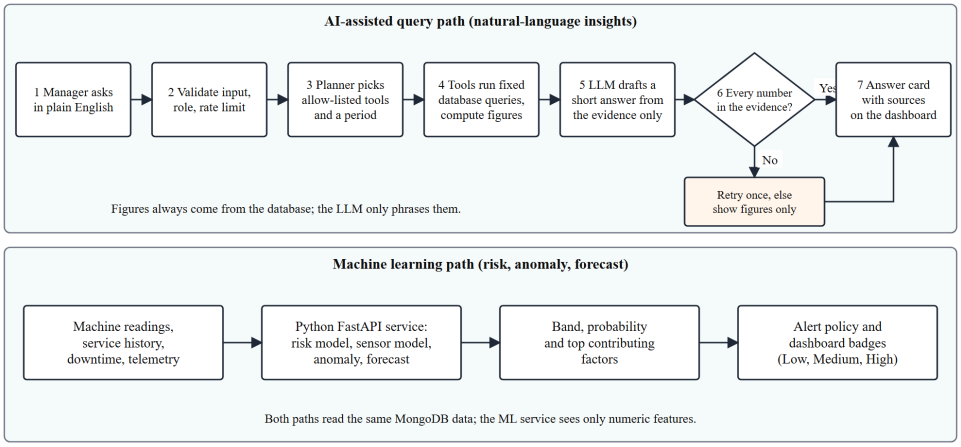


Figure 4.8: AI/ML workflow – query path and machine-learning path

LLM-based intelligence layer

1. The manager types a question and chooses a period; the request is validated, role-checked and rate-limited.
2. A query planner chooses up to three tools from a fixed allow-list (machine status, downtime, production, inventory, attendance, saved assessments, work orders, reliability, stock and order forecasts). It cannot name a collection or write a query.
3. Each tool runs a fixed database query and returns bounded, already-calculated key figures.
4. Only those figures are sent to the language model, which writes a short answer. Text from the database is treated as data, never as instructions.
5. Every number in the answer must appear in the evidence. If not, the call is retried once, and then only the figures are shown.

Machine-learning service

- **Risk model:** logistic regression chosen by cross-validation over a calibrated random forest; six inputs (run hours, days since service, downtime events, tool wear, temperature, vibration); trained on synthetic data and labelled as such everywhere it is shown.
- **Sensor model:** gradient boosting trained on the public AI4I 2020 dataset using rpm, torque (derived from power) and tool wear.
- **Anomaly and forecast:** an Isolation Forest with a robust z-score learns each machine's own recent telemetry; a trend model forecasts daily stock use and order output with an early and a late estimate.

CHAPTER 5: IMPLEMENTATION

5.1 Implementation Overview

SIMS is implemented as three programs that run side by side: the **API server** (Node.js with Express, Mongoose and Socket.io, default port 5000), the **web client** (React with TypeScript, built with Vite) and the **ML service** (Python with FastAPI). They are started independently; the client talks to the API through a development proxy or a configured API address, and the API talks to the ML service over HTTP with a timeout. The seed script fills a development database with ten machines, four workers, eight inventory items and a few orders so that every screen has data on first run, and refuses to run against a non-local database unless explicitly confirmed.

The server code is split into one folder per module (access, machines, workers, attendance, production, inventory, alerts, analytics, maintenance, purchasing, telemetry, leave, notifications, planning, insights, simulator, AI and ML). A simulator, available to administrators, advances machine status, readings and order output on a timer through the same service functions used by real requests; it was used to generate the data seen in the screenshots in this chapter.

5.2 Authentication and Role-Based Access

Sign-in verifies the password hash with bcrypt and returns a 15-minute access token (JWT) in the response body and a seven-day refresh token in an httpOnly cookie; the refresh token is rotated on every use and only its hash is stored. On every request the middleware verifies the token and then looks up the user’s current role and active flag (cached for a few seconds), so deactivating an account or changing a role takes effect immediately and a token issued before a password reset is refused. The `authorize` guard is added to each route that is restricted, and the same check runs when a Socket.io connection is made.

server/src/middleware/auth.js

```
export async function authenticate(req, _res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return next(ApiError.unauthorized('Missing access token'));

  let payload;
  try {
    payload = jwt.verify(token, SIGNING_KEY);
  } catch {
    return next(ApiError.unauthorized('Invalid or expired access token'));
  }
  try {
    req.user = await resolveUser(payload);
    next();
  } catch (err) {
    next(err);
  }
}

export const authorize = (...roles) => (req, _res, next) => {
  if (!req.user) return next(ApiError.unauthorized());
  if (!roles.includes(req.user.role)) return next(ApiError.forbidden('Insufficient permissions'));
  next();
};
```

Figure 5.1: JWT authentication middleware and role guard (server/src/middleware/auth.js)

Workers receive further, data-level restrictions inside the services: for example a worker can list only the work orders and orders connected to their own tasks, and can change only the status of tasks assigned to them. These rules are covered by tests that sign in as each role and assert the status code for each protected route.

5.3 Screenshots of the Application

All screenshots were taken from the running application against a database filled by the seed script and the simulator; none are mock-ups.

5.3.1 Sign-in and dashboard

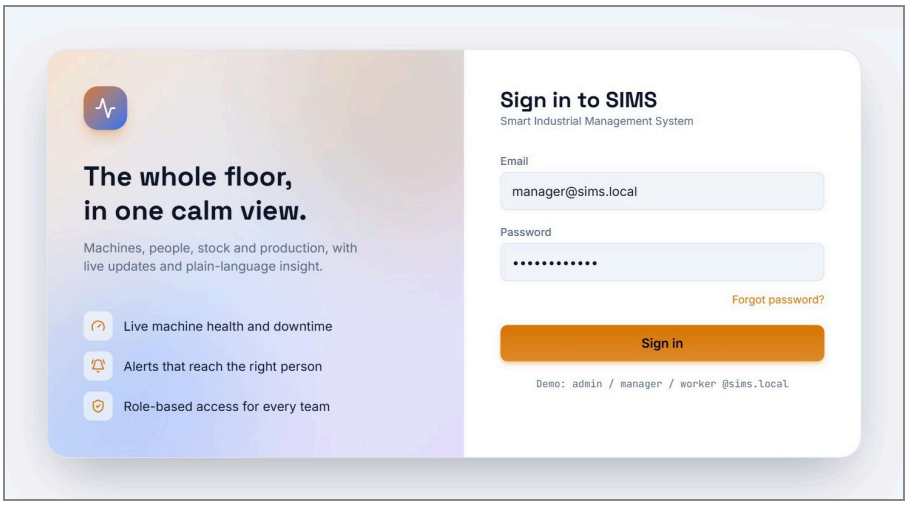


Figure 5.2: Sign-in page (demo accounts are pre-filled only in development builds)

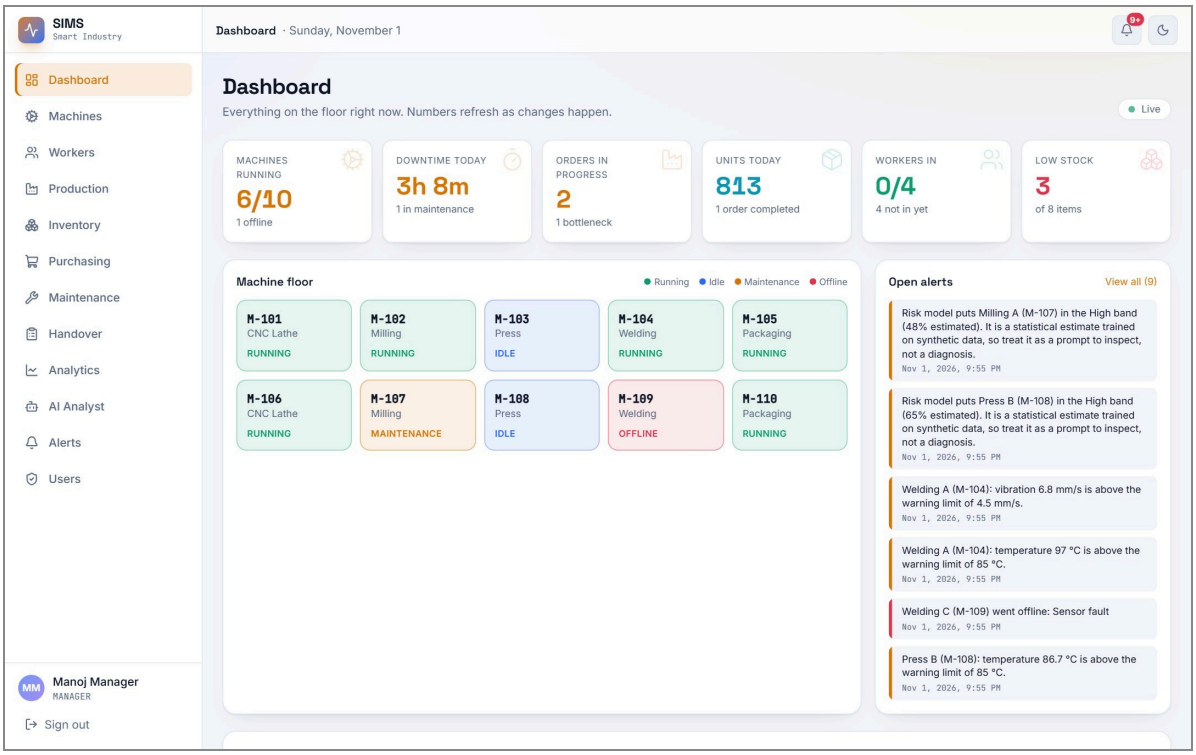


Figure 5.3: Manager dashboard – KPI tiles, live machine floor and open alerts

The dashboard shows six KPI tiles, the status of every machine as colour-coded tiles, and the open alerts. The colour of the first tile depends on how many machines are running. Everything refreshes from live events; a dark theme is also available from the toggle in the top bar.

5.3.2 Machine monitoring

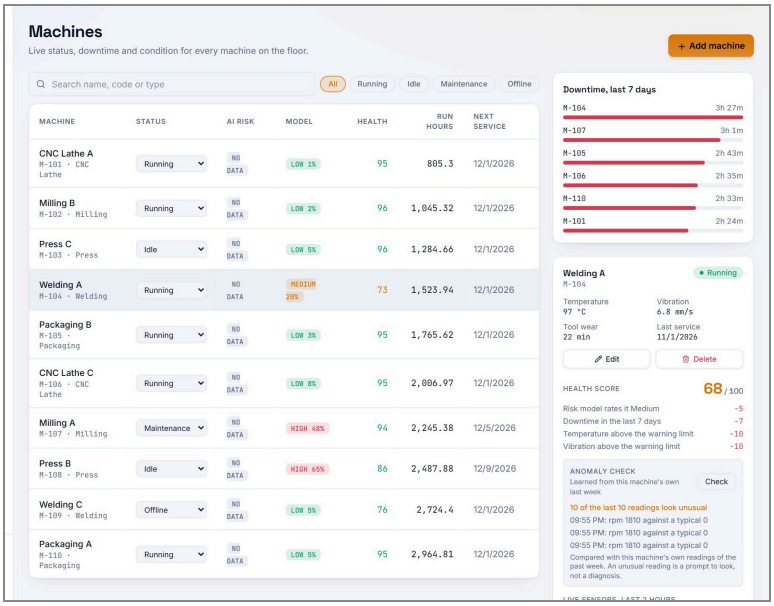


Figure 5.4: Machines – status, AI risk, ML model band, health score and next service; detail panel with anomaly check

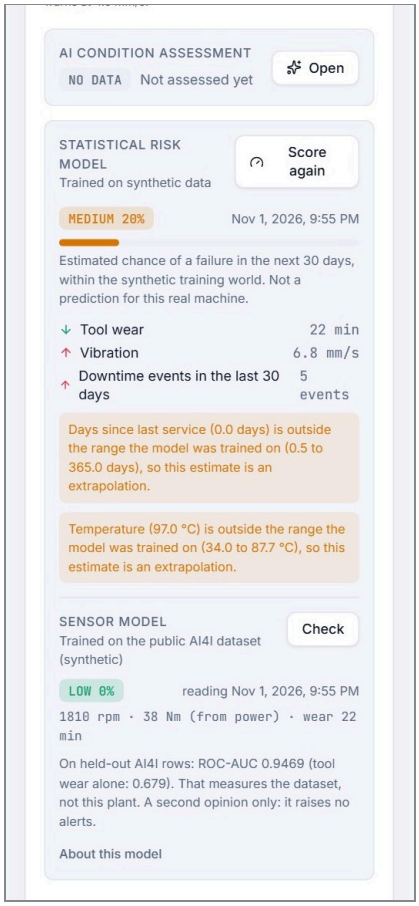


Figure 5.5: Risk panel of a machine

The panel under the sensor chart shows two independent signals. The **statistical risk model** reports a band (here Medium, 20%) with the factors that raised or lowered it, and warns when an input lies outside the range the model was trained on. The **sensor model** gives a second opinion from rpm, torque and tool wear and states that it is trained on a public dataset.

Both are labelled as estimates; neither raises an alert on its own authority beyond the stated policy: a High band raises one warning, and a Low band clears it.

5.3.3 Worker management

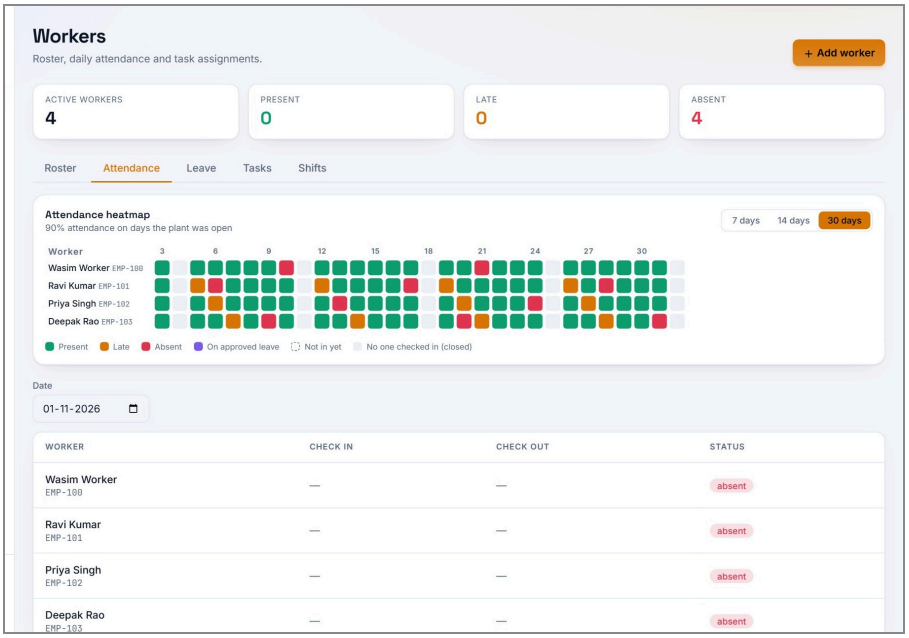


Figure 5.6: Workers – 30-day attendance heatmap and the day's attendance table

Each square is one worker on one day: green for present, orange for late, red for absent, purple for approved leave and grey for a day nobody checked in (the plant was closed). Closed days are therefore not counted against anyone.

5.3.4 Production tracking

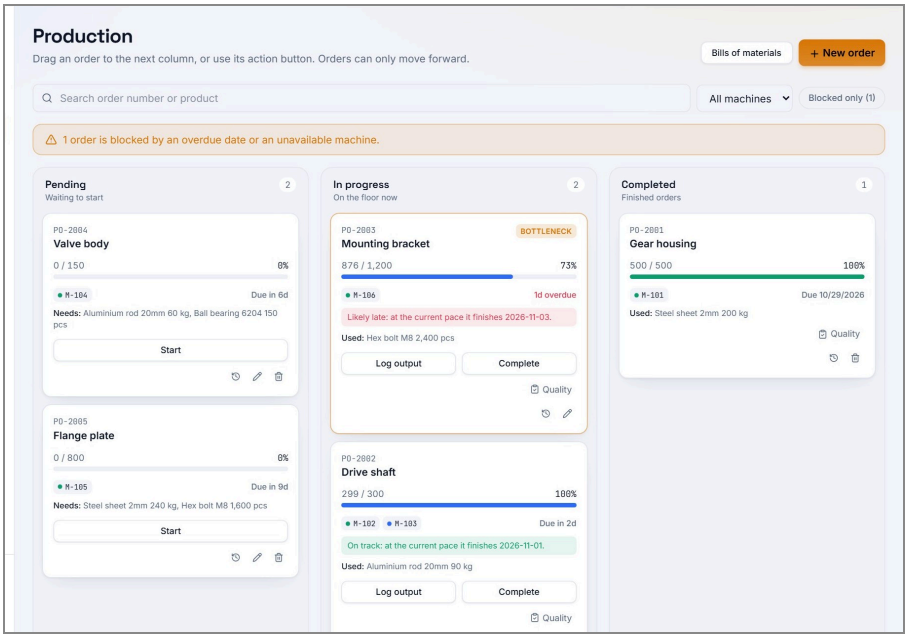


Figure 5.7: Production – kanban board with a bottleneck flag and projected finish for each order

5.3.5 Inventory control and order feasibility

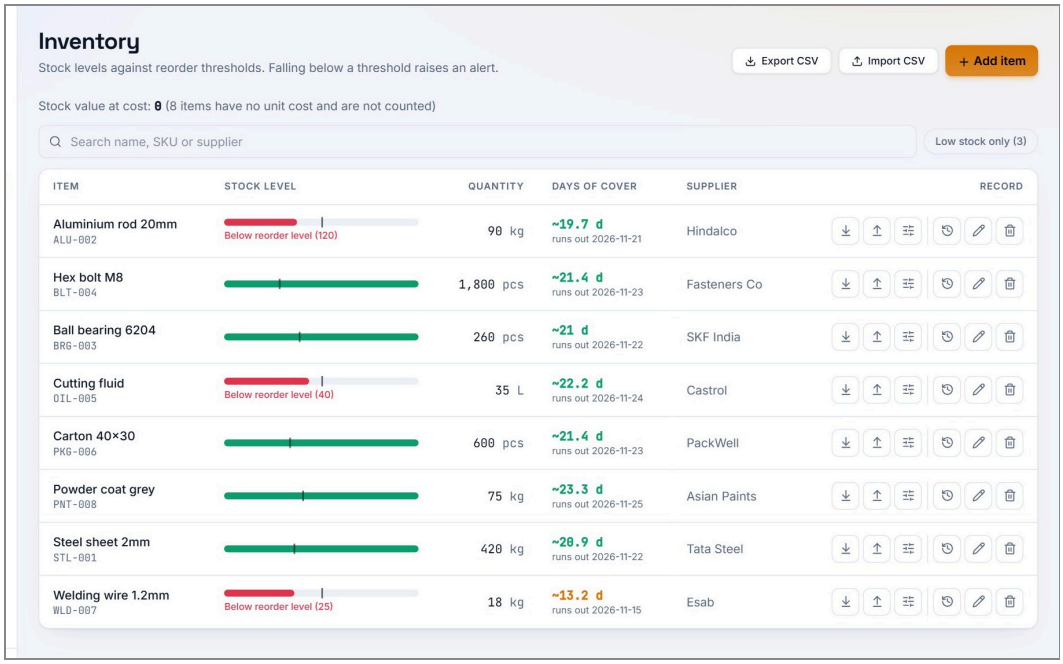


Figure 5.8: Inventory – stock bars against reorder levels, days of cover forecast and CSV tools

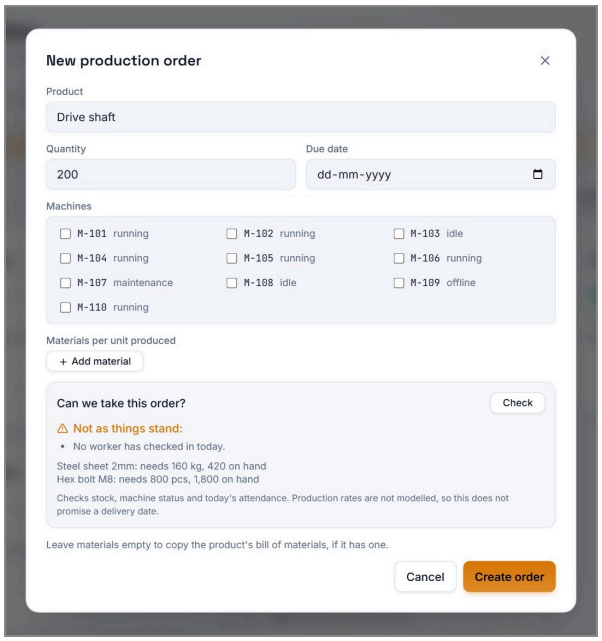


Figure 5.9: New production order with the feasibility check

Creating an order shows a deterministic check, “Can we take this order?”, that compares the bill of materials with stock, the chosen machines with their status, and the day’s attendance. In this capture it reports that no worker has checked in yet and lists the materials needed against what is on hand.

The check does not promise a delivery date because production rates are not modelled; the screen says so.

5.4 Source Code of Some Modules

The excerpts below are taken from the project source; line numbers are those of the source files, and the token signing key is shown as a placeholder.

5.4.1 Machine status change and real-time events

`changeStatus` is the heart of machine monitoring. It refuses a change to the same status, closes the open downtime log with its duration, opens a new one, saves the machine and then emits the event to all clients and to the machine's own room. Going offline raises a critical alert and opens a corrective work order; coming back resolves the alert.

server/src/modules/machines/machine.service.js

```
54  if (machine.status === status) throw ApiError.conflict(`Machine is already ${status}`);
55
56  const now = new Date();
57  const open = await MachineLog.findOne({ machine: id, endedAt: null }).sort({ startedAt: -1 });
58  if (open) {
59    open.endedAt = now;
60    open.durationMins = Math.max(0, Math.round((now - open.startedAt) / 60_000));
61    await open.save();
62  }
63  const from = machine.status;
64  await MachineLog.create({ machine: id, fromStatus: from, toStatus: status, startedAt: now, reason });
65  machine.status = status;
66  await machine.save();
67
68  const payload = { machineId: String(id), code: machine.code, name: machine.name, from, to: status, at: now };
69  emit('machine:statusChanged', payload);
70  emit('machine:statusChanged', payload, `machine:${id}`);
    ...
72  if (status === 'offline') {
73    await alerts.create({
74      type: 'machine_offline', severity: 'critical', source: 'machines', sourceId: id,
75      message: `${machine.name} (${machine.code}) went offline${reason ? `: ${reason}` : '.'}`,
76    });
77    await maintenance.openForDowntime(machine, reason);
78  } else if (from === 'offline') {
79    await alerts.resolveFor('machine_offline', id);
80  }
```

Figure 5.10: Machine status service (server/src/modules/machines/machine.service.js)

5.4.2 Socket.io handshake and rooms

server/src/sockets/index.js

```
12  io.use(async (socket, next) => {
13    try {
14      const payload = jwt.verify(socket.handshake.auth?.token, SIGNING_KEY);
15      // The same rules as the REST API: active account, current role, token not older than a password reset.
16      socket.data.user = await resolveUser(payload);
17      next();
18    } catch {
19      next(new Error('Unauthorized'));
20    }
21  });
22
23  io.on('connection', (socket) => {
24    const { role } = socket.data.user;
25    socket.join('room:all');
26    socket.join(`room:${role}`);
27    socket.join(`user:${socket.data.user.id}`);
28    if (role !== 'worker') socket.join('room:staff');
    ...
60  export function emit(event, payload, ...rooms) {
61    if (!io) return;
62    const targets = rooms.length ? rooms : ['room:all'];
63    targets.reduce((chain, room) => chain.to(room), io).emit(event, payload);
```

Figure 5.11: Authenticated socket connection, rooms and the emit helper (server/src/sockets/index.js)

5.4.3 Atomic stock transaction

The quantity guard is part of the update filter, so two simultaneous stock-outs cannot both succeed when only one is covered by the stock.

server/src/modules/inventory/inventory.service.js

```
export async function addTransaction(id, { type, qty }, userId) {
  const delta = type === 'out' ? -qty : qty;
  const filter = { _id: id };
  if (delta < 0) filter.quantity = { $gte: -delta };

  const item = await InventoryItem.findOneAndUpdate(filter, { $inc: { quantity: delta } }, { new: true });
  if (!item) {
    if (!(await InventoryItem.exists({ _id: id }))) throw ApiError.notFound('Inventory item not found');
    throw ApiError.conflict('Insufficient stock for this transaction');
  }

  const tx = await StockTransaction.create({ item: id, type, qty, by: userId });
  await afterChange(item);
  return { item, transaction: tx };
}
```

Figure 5.12: Guarded stock update (server/src/modules/inventory/inventory.service.js)

5.4.4 Production order state machine

server/src/modules/production/order.service.js

```
const TRANSITIONS = { pending: ['in_progress'], in_progress: ['completed'], completed: [] };
...
  if (!TRANSITIONS[order.status].includes(next)) {
    throw ApiError.conflict(`Cannot move an order from ${order.status} to ${next}`);
  }
...
  if (next === 'in_progress') {
    const down = order.machines.filter((m) => DOWN.includes(m.status));
    if (down.length) throw ApiError.conflict(`Machines unavailable: ${down.map((m) => `${m.code} (${m.status})`).join(', ')}`);
    if (order.materials.length && !order.materialsConsumed) {
      consumption = await consumeForOrder(order, userId);
      order.materialsConsumed = true;
    }
```

Figure 5.13: Allowed order transitions and the start-order checks (server/src/modules/production/order.service.js)

5.4.5 Machine-learning integration

server/src/modules/ml/ml.service.js

```
export async function buildFeatures(machine) {
  const e = await buildEvidence(machine, { includeHistory: true });
  const features = {
    runHours: e.readings.runHours?.value,
    daysSinceService: e.service.daysSinceLastService?.value,
    downtimeCount30d: e.history?.downtimeEvents.value,
    toolWearMin: e.readings.toolWearMin?.value,
    temperatureC: e.readings.temperatureC?.value,
    vibrationMms: e.readings.vibrationMms?.value,
  };
...
  const result = await client.predict(features);
  const saved = await MlPrediction.create({
    machine: machine._id, risk: result.risk, probability: result.probability, features, featuresHash,
    topFactors: result.topFactors, warnings: [...dataWarnings, ...result.warnings], modelVersion: result.modelVersion,
    dataSource: result.dataSource, source, createdBy: userId, predictedAt: new Date(),
  });

  await applyAlertPolicy(machine, result);
  const updated = await Machine.findByIdAndUpdate(machine._id, { mlRisk: result.risk, mlProbability: result.probability,
mlScoredAt: saved.predictedAt }, { new: true }).lean();
  emit('machine:updated', updated, 'room:all');
```

Figure 5.14: Building features and scoring through the ML service (server/src/modules/ml/ml.service.js)

5.4.6 Grounding check for AI answers

The language model is never trusted with figures. After it writes an answer, every number in the text is compared with the numbers present in the evidence that was supplied; a mismatch triggers a retry and then a fallback to the plain figures.

server/src/modules/ai/analyst.service.js

```
const NUMBER = /\d[\d,]*(?:\.\d+)?/g;
const numbersIn = (s) => (s.match(NUMBER) ?? []).map((n) => n.replace(/,/g, ''));

/** Numbers in the prose that appear nowhere in the evidence the model was given. */
export function ungroundedNumbers(answer, evidenceJson) {
  const allowed = new Set(numbersIn(evidenceJson));
  return [...new Set(numbersIn(answer).filter((n) => !allowed.has(n)))];
}
```

Figure 5.15: Detecting numbers that are not in the evidence (server/src/modules/ai/analyst.service.js)

5.4.7 Client live updates

client/src/hooks/useLiveUpdates.ts

```
    // Any change can move a KPI, so cached analytics are refreshed alongside the data that changed.
    const refresh = (...keys: string[]) => [...keys, 'analytics'].forEach((k) => qc.invalidateQueries({ queryKey: [k] }));

    socket.on('machine:statusChanged', (p: { name: string; code: string; to: string }) => {
      refresh('machines', 'downtime');
      push({ tone: p.to === 'offline' ? 'critical' : 'info', title: `${p.code} is now ${p.to}`, body: p.name, keep: true });
    });
    socket.on('machine:created', () => refresh('machines'));
    socket.on('machine:updated', () => refresh('machines'));
    socket.on('machine:deleted', () => refresh('machines'));
    socket.on('inventory:updated', () => refresh('inventory'));
```

Figure 5.16: Socket events that refresh the client cache (client/src/hooks/useLiveUpdates.ts)

5.5 AI/ML Integration Summary

Table 5.1: AI and ML features, where they appear and what they use

Feature	Where it appears	Data used
Failure-risk band and probability	Machines page; Medium and High raise or clear one alert	Run hours, days since service, downtime count, tool wear, temperature, vibration
Sensor-model second opinion	Machines page (on demand)	Newest rpm, power-derived torque, tool wear
Anomaly check	Machines page (on demand)	A machine’s own last seven days of telemetry
Days of cover; projected order finish	Inventory, Production, Dashboard	30 days of stock use; 14 days of logged output
Natural-language analyst; weekly summary; alert explanations	AI Analyst, Alerts	Figures retrieved by fixed queries; never raw collections

CHAPTER 6: TESTING AND RESULTS

6.1 Testing Methodology

Testing was done at three levels. **Unit tests** cover pure rules such as the state machine, the health score, the feature builder, grounding and forecasts. **Integration tests** start the real Express application against an in-memory MongoDB and call the REST API with each role, so validation, authorisation, side effects and the response envelope are tested together. **Service tests** for the ML service call the FastAPI application through its test client and check that models load, inputs are validated and outputs are in range. The AI layer is tested with a stub in place of the language model so that tests are repeatable; its quality is measured separately (Section 6.4).

6.2 Automated Test Results

All suites were run on 1 November 2026. Every test passed; the counts below are taken from the test runners, not estimated.

Table 6.1: Automated tests by area

Suite	Area covered	Tests	Passed
Server	Authentication, lockout, password reset, users, configuration, API documentation	47	47
Server	Machines, maintenance, telemetry, simulator	29	29
Server	Workers, attendance, leave, tasks	26	26
Server	Production, bill of materials, inventory, purchasing	30	30
Server	Analytics	11	11
Server	ML integration and insights	38	38
Server	Language-model layer: analyst, assessment, explanation, limits, evaluation harness	88	88
Client	13 files: CSV tools, formatting, stores, components, guards	113	113
ML service	Risk model, sensor model, anomaly and forecasts (3 files)	56	56
Total		**438**	**438**

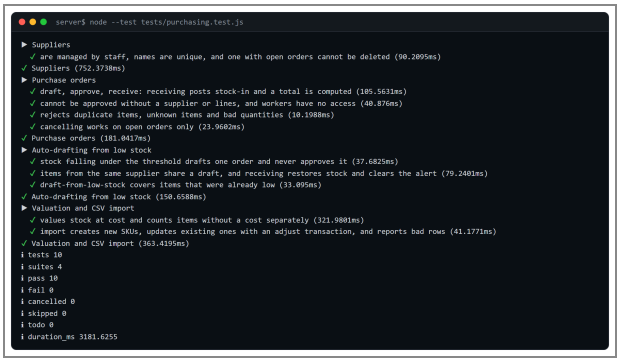


Figure 6.1: Test runner output (purchasing)

The figure shows the real output of the purchasing suite. Besides the rules of purchase orders (receiving stock, partial receipts, cancellation) it asserts that a worker cannot reach the routes and that invalid bodies are rejected with field-level errors.

The pytest output below it shows the twelve tests of the insights endpoints of the ML service.



Figure 6.2: pytest output (ML insights)

6.3 Sample Test Cases

Table 6.2: Selected functional test cases

ID	Scenario	Expected result	Result
TC-01	Sign in with a wrong password five times	Account is locked; the sixth attempt is refused even with the right password	Pass
TC-02	Worker calls a manager-only route	HTTP 403 and no data change	Pass
TC-03	Change a machine to the status it already has	HTTP 400, no new downtime log	Pass
TC-04	Set a machine offline, then running again	Critical alert and work order opened, then alert resolved; downtime duration recorded	Pass
TC-05	Stock-out larger than the quantity on hand	HTTP 400, quantity unchanged	Pass
TC-06	Start an order with insufficient materials	Order stays pending; shortage is listed	Pass
TC-07	Move an order from completed back to pending	Rejected by the state machine	Pass
TC-08	Receive a purchase order partly, then fully	Stock increases by each receipt; status changes partial → received	Pass
TC-09	Analyst answer contains a number not in the evidence	Retry once, then the figures only are shown	Pass
TC-10	ML service is stopped	Machines page loads; the model panel shows that the service is unavailable	Pass

6.4 Measured Performance

Response times were measured on the development machine with the API and the database on the same host, using 40 sequential requests for each endpoint after sign-in. The figures show the speed of the code on a small data set and are **not** a load test; production behaviour will differ.

Table 6.3: API response time, 40 requests each (milliseconds)

Endpoint	Median	95th percentile	Maximum
GET /api/machines	4.5	7.4	7.9
GET /api/orders	9.4	11.6	13.9
GET /api/inventory	3.9	5.7	6.6
GET /api/analytics/overview	7.0	11.8	12.4
GET /api/analytics/downtime	5.4	9.7	10.4
GET /api/alerts	4.7	8.8	15.4

For the real-time channel, 20 status changes were made through the API while a second connected client listened. All 20 events were delivered; the time from the request to the event arriving at the client had a median of 7.8 ms and a maximum of 13.3 ms.

6.5 Evaluation of the ML and AI Components

Table 6.4: Held-out evaluation of the learned models

Model	Data	ROC-AUC	PR-AUC	Brier (base rate)
Risk model (logistic regression)	Synthetic, 18,000 training samples	0.801	0.428	0.0707 (0.0881)
Sensor model (gradient boosting)	Public AI4I 2020, 10,000 rows, 339 failures	0.947	0.591	0.0192 (0.0328)

- **Risk model.** The ROC-AUC of 0.801 is almost equal to the 0.8015 ceiling of the generator that produced the data, so the model has learned what the data contains; it says little about real machines. The High band (probability ≥ 0.30) found 35% of the failures with 52% precision. Applied to the AI4I data, where only two of its inputs exist, it scored 0.662, the same as tool wear alone, which is why it is described as a demonstration of the pipeline.
- **Sensor model.** Tool wear alone gives 0.679 on the same split, so rpm and torque add real information. This model comes from a public data set of a milling process and is shown as a second opinion.
- **Language-model layer.** In a live evaluation of 42 analyst questions, 39 were answered (the others hit service limits); the correct tool was chosen in 29 of 29 questions that had a defined tool; the key figures matched an independently computed answer in 28 of 29; no answer contained forbidden content. For assessments, 6 scenarios were run three times each: all 18 outputs were valid in form and 15 were rated acceptable by the checks; after the fleet context was added to the prompt, all 14 that completed were acceptable (4 timed out).

6.6 Security Checks

- Role guards were tested route by route: a worker receives HTTP 403 on manager-only routes, and an unauthenticated request receives HTTP 401.
- Five failed sign-ins lock the account for a period; the lock is covered by a test and cannot be bypassed by the correct password.
- Bodies are validated with schemas; invalid fields are rejected with field-level messages before any database access.

6.7 Frontend and Usability Checks

Every page was opened in the light and dark themes at a 1440-pixel desktop width and at a narrow width to confirm that the menu collapses, tables scroll inside their container and dialogs can be closed with the keyboard. The dialog component traps focus, closes on Escape and labels its title for screen readers, and a console check found no errors or warnings while moving through all pages as a manager.

CHAPTER 7: CONCLUSION AND SUMMARY

7.1 Summary of Work

This project designed and built SIMS, a centralised web platform that brings machines, workers, production and inventory into one system with live updates. The application tier exposes 132 documented API operations over 26 MongoDB collections; the client has role-specific pages for administrators, managers and workers; and a separate ML service and an LLM-based intelligence layer add prediction and natural-language insight. The code base holds about 5,900 lines on the server, 5,900 in the client and 800 in the ML service, tested by 438 automated tests.

7.2 Objectives Achieved

- A single dashboard with live machine status, KPIs and alerts that update without a page refresh.
- Machine monitoring with downtime logging, telemetry, health score, maintenance scheduling and reliability measures (MTBF and MTTR).
- Worker management with shifts, attendance, leave, tasks and handover notes, and a personal page for each worker.
- Production tracking with a controlled order life cycle, bill of materials, quality checks and an order feasibility check.
- Inventory control with atomic transactions, reorder alerts, valuation, suppliers and purchase orders.
- AI-powered operational insights whose figures are checked against the evidence, with machine-learning predictions that always state their data and limits.

7.3 Implemented and Proposed Features

Table 7.1: Status of the features proposed in the synopsis

Feature	Status
Dashboard, five modules, role-based access	Implemented and tested
Real-time updates over Socket.io	Implemented; latency measured locally
Failure-risk model and anomaly check	Implemented; trained on synthetic and public data only
LLM-based analyst, summaries and alert explanations	Implemented; quality measured on a fixed question set
Physical sensor integration, mobile application, hosted deployment	Not implemented (see Chapter 8)

7.4 Learning Outcomes

The team learned to model a real-time system with Socket.io rooms; to keep side effects in services so that every entry point behaves the same; to use atomic database operators for counters; to evaluate machine-learning models honestly, including an external check that lowered our own result; and to constrain a language model so that it can explain data but not invent it.

CHAPTER 8: LIMITATIONS AND FUTURE SCOPE

8.1 Limitations

- **Synthetic risk-model data.** The failure-risk model is trained on generated data. Its accuracy on real machines is unknown until labelled history from a plant is available.
- **No real sensors.** Telemetry is produced by a simulator or entered through the API; there is no connection to PLCs or industrial protocols such as OPC UA or MQTT.
- **Not deployed or load-tested.** The system has been run only in development. A deployment description exists, but no hosted instance, concurrency or long-run test was performed.
- **Single plant, single language.** There is no multi-tenant or multi-site support, and the interface is in English only.
- **No production rates.** Orders carry quantities and due dates but not cycle times, so the feasibility check cannot promise a delivery date.
- **AI service limits.** The language-model features depend on an external service; under rate limits or outages the interface shows that insight is unavailable rather than a guess.

8.2 Privacy and Ethical Considerations

The intelligence layer receives only figures that the application has already calculated; names, passwords, tokens and raw records are not sent to the language model. Predictions are always shown as estimates with their data source, and no model can approve a purchase, change a status or edit a record. Decisions therefore remain with the people who use the system.

8.3 Future Scope

- Connect machines through MQTT or OPC UA gateways and retrain the risk model on real labelled failures.
- Add mobile applications or a progressive web app with push notifications for supervisors.
- Add shift-wise production planning, energy monitoring and quality charts (SPC).
- Add multi-plant support, per-plant roles and scheduled report e-mails.
- Containerise the three programs and add monitoring, backups and a continuous-integration pipeline for hosted deployment.

BIBLIOGRAPHY

[1] MongoDB Inc., “MongoDB Manual – Data Modeling and Aggregation,” https://www.mongodb.com/docs/manual/.

[2] OpenJS Foundation, “Express – Node.js web application framework,” https://expressjs.com/.

[3] Meta Platforms Inc., “React Documentation,” https://react.dev/.

[4] Socket.IO, “Socket.IO Documentation,” https://socket.io/docs/v4/.

[5] Mongoose, “Mongoose Documentation,” https://mongoosejs.com/docs/.

[6] S. Ramírez, “FastAPI Documentation,” https://fastapi.tiangolo.com/.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] S. Matzka, “Explainable Artificial Intelligence for Predictive Maintenance Applications,” in Proc. 3rd Int. Conf. on Artificial Intelligence for Industries (AI4I), 2020. Data set: UCI Machine Learning Repository, “AI4I 2020 Predictive Maintenance Dataset.”

[9] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” in Proc. IEEE Int. Conf. on Data Mining (ICDM), 2008, pp. 413–422.

[10] Zod, “Zod – TypeScript-first schema validation,” https://zod.dev/.

[11] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” https://owasp.org/www-project-top-ten/.

[12] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.

[13] TanStack, “TanStack Query Documentation,” https://tanstack.com/query/latest.

[14] Recharts, “Recharts – a composable charting library for React,” https://recharts.org/.

[15] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed., McGraw-Hill, 2020.